

Operaciones con vectores

Mario A. Morales Rivera

`mamorales@correo.unicordoba.edu.co`

Universidad de Córdoba



Contenido I

Operaciones con vectores



Operaciones con vectores



¿Qué aprenderemos?

- Comprender el concepto de vectorización en R
- Comprender qué son los vectores lógicos y cómo manipularlos.
- Usar la indexación lógica para seleccionar subconjuntos de un vector.
- Ser capaz de utilizar valores infinitos y perdidos.



Operaciones con vectores

Los vectores son muy útiles porque permiten a R realizar operaciones en múltiples elementos simultáneamente, con velocidad y eficiencia. Este comportamiento orientado a vectores, vectorizado o de elementos es una característica clave del lenguaje.



```
1:5
```

```
[1] 1 2 3 4 5
```

```
#class(1:5)
```

```
#mode(1:5)
```

```
6:10
```

```
[1] 6 7 8 9 10
```

```
1:5 + 6:10 # vectorizacion
```

```
[1] 7 9 11 13 15
```



Operación Suma

La operación suma de dos vectores se hace componente a componente

```
c(1, 3, 6, 10, 15) + c(0, 1, 3, 6, 10)
```

```
[1] 1 4 9 16 25
```



Operación Resta

La operación resta de dos vectores se hace componente a componente

```
c(1, 3, 6, 10, 15) - c(0, 1, 3, 6, 10)
```

```
[1] 1 2 3 4 5
```



Operación producto

La operación producto de dos vectores se hace componente a componente

```
c(1, 3, 6, 10, 15) * c(0, 1, 3, 6, 10)
```

```
[1] 0 3 18 60 150
```



Operación cociente

La operación cociente de dos vectores se hace componente a componente

```
c(1, 3, 6, 10, 15) / c(0, 1, 3, 6, 10)
```

```
[1]      Inf 3.000000 2.000000 1.666667 1.500000
```



Reciclado de valores

Cuando los vectores no tienen la misma longitud, el vector más corto se replica hasta alcanzar la longitud del más largo. En caso de que la longitud del más largo no sea un múltiplo de la longitud del más corto manda un mensaje de advertencia.

```
c(1, 3, 6, 10, 15,17)+c(3,6,8)
```

```
[1]  4  9 14 13 21 25
```



Reciclado de valores (II)

```
# mensaje de advertencia  
c(1, 3, 6, 10, 15, 17)+c(3, 6, 8, 10)
```

```
[1]  4  9 14 20 18 23
```

```
Warning message in c(1, 3, 6, 10, 15, 17) + c(3, 6, 8, 10):  
"longitud de objeto mayor no es múltiplo de la longitud de uno menor"
```



Resta

```
foo<-c(1, 3, 6, 10, 15,17)  
bar<-c(3,6,8)  
foo-bar
```

```
[1] -2 -3 -2  7  9  9
```



¿Cómo trabaja?

```
x=rep(-1,length(foo))
plot(x,1:length(foo),axes=FALSE,ann=FALSE,type="p",col='white',xlim=c(-2,2),ylim=c(0,1))
arrows(x-0.5,length(foo):1,x+0.5,length(foo):1,code=3,angle=20,length=0.5)
text(x-0.6,length(foo):1,labels=foo)
text(x-0.8,length(foo):1,labels=paste('[',1:length(foo),']',sep=''))
text(x+0.6,length(foo):1,labels=bar)
text(x+0.8,length(foo):1,labels=paste('[',1:length(bar),']',sep=''))
text(x+0.8,length(foo):1,labels=paste('[',1:length(bar),']',sep=''))
mtext("foo",side=3, at=-1-0.8)
mtext("bar",side=3, at=-1+0.8)
mtext("_",side=3, at=-1)
```

cuando la longitud del más corto no es divisor de la long del mas largo

```
x=rep(1,length(foo))
bar<-c(3,6,8,4)
```

```
arrows(x-0.5,length(foo):1,x+0.5,length(foo):1,code=3,angle=20,length=0.5)
```



Vectorización

- En las operaciones anteriores se ha usado el concepto de *vectorización*, el cual tiene varios significados en R, el más común de los cuales es que

! Importante

Un operador o una función actuará sobre cada elemento de un vector sin la necesidad de que usted escriba un bucle.



Vectorización: segundo significado

- Un segundo significado de *vectorización* es cuando una función toma un vector como entrada y calcula una estadística de resumen.

```
# primer concepto de vectorización  
x=c(3,5,1,7,4)  
# raiz usando vectorizacion  
sqrt(x)
```

```
[1] 1.732051 2.236068 1.000000 2.645751 2.000000
```

```
sqrt(c(4,9,16,1))
```

```
[1] 2 3 4 1
```



Sin Vectorización (no recomendado)

```
# raiz usando un bucle for
# no recomendado
for(i in x){
  print(sqrt(i))
}
```

[1] 1.732051

[1] 2.236068

[1] 1

[1] 2.645751

[1] 2



Segundo concepto de vectorizacion

```
# segundo concepto de vectorizacion  
# la función se aplica sobre un vector  
sum(1:5)
```

```
[1] 15
```

```
sum(x)
```

```
[1] 20
```

```
median(1:5)
```

```
[1] 3
```

```
median(x)
```

```
[1] 4
```



Ejemplos de vectorización

Todos los operadores aritméticos en R están vectorizados. Los siguientes ejemplos demuestran la resta, la multiplicación, la exponenciación y dos tipos de división, así como el residuo de la división:

```
c(2, 3, 5, 7, 11, 13) - 2 #subtracción
```

```
[1] 0 1 3 5 9 11
```

```
-2:2 * -2:2 #multiplicación
```

```
[1] 4 1 0 1 4
```

```
1:10 / 3 ## división usual (de punto flotante)
```

```
[1] 0.3333333 0.6666667 1.0000000 1.3333333 1.6666667 2.0000000 2.3333333  
[8] 2.6666667 3.0000000 3.3333333
```

```
1:10 %/% 3 ## división entera
```

```
[1] 0 0 1 1 1 2 2 2 3 3
```



Ejemplo de aplicación

! Importante

¿Cuántos números del 1 al 10 son divisibles por 3?

```
1:10 %% 3
```

```
[1] 1 2 0 1 2 0 1 2 0 1
```

```
1:10 %% 3 == 0
```

```
[1] FALSE FALSE  TRUE FALSE FALSE  TRUE FALSE FALSE  TRUE FALSE
```

```
sum(1:10 %% 3 == 0 )
```

```
[1] 3
```

```
# cuantos no divisibles por 3
```

```
#sum(1:10%% 3!=0 )
```

```
#(1:10)[1:10%% 3 == 0]
```



Indexación lógica

```
x=c(3,5,1,7,4)  
x[c(TRUE,FALSE,TRUE,FALSE,FALSE)]
```

```
[1] 3 1
```

! Importante

¿Cuáles números del 1 al 10 son divisibles por 3?

```
(1:10)[1:10 %% 3 == 0]
```

```
[1] 3 6 9
```



Cálculos por niveles de un factor

! Importante

Datos de personas, sexo y edad,

- ¿Cuántas personas de cada sexo hay?
- Calcular la media de edad para cada sexo.

```
sexo=c("M","M","F","M","M","F","F","F","M")
```

```
edad=c(18,15,16,18,17,15,20,19,15)
```



Solución

- ¿Cuántos hay?

```
sum(sexo=="M")
```

```
[1] 5
```

```
sum(sexo=="F")
```

```
[1] 4
```



Media de edad para cada sexo

```
mean(edad[sexo=="M"])
```

```
[1] 16.6
```

```
mean(edad[sexo=="F"])
```

```
[1] 17.5
```

```
mean(edad[sexo!="M"])
```

```
[1] 17.5
```



Ejercicio

Practicar el cálculo de las funciones trigonométricas, exponencial, logaritmica, usando el concepto de vectorización.



Operadores de comparación

Para determinar si dos valores enteros son iguales se usa `==` (no use `=` recuerde que es para asignar). Los operadores relacionales son:

- `==` igual a, idéntico a
- `!=` distinto a, diferente a , no igual a
- `>` Mayor que
- `>=` Mayor o igual que
- `<` Menor que
- `<=` Menor o igual que



Operadores de comparación II

Todos estos operadores son vectorizados, veamos algunos ejemplos.

```
c(3, 4 - 1, 1 + 1 + 1) == 3
```

```
[1] TRUE TRUE TRUE
```

```
l=c(3, 4 - 1, 1 + 1 + 1) == 3  
class(l)
```

```
[1] "logical"
```

El resultado de este operador es un vector lógico, es decir, un vector cuyas entradas son TRUE o FALSE , estudiaremos en detalle estos vectores más adelante.



```
1:3 != 3:1 # diferente
```

```
[1] TRUE FALSE TRUE
```



```
exp(1:5) < 100 # menor que
```

```
[1] TRUE TRUE TRUE TRUE FALSE
```



Problema de aplicación

! Importante

Un grupo de estudiantes obtuvo las siguientes notas 3.5,4.1,2.0,3.0,1.5,4.5,3.9 escribir código R que me diga cuantos estudiantes tienen una nota mayor o igual que el promedio de nota del curso.



Solución

```
notas=c(3.5,4.1,2.0,3.0,1.5,4.5,3.9)
mean(notas)
```

```
[1] 3.214286
```

¿Cuántos estudiantes tienen una nota mayor o igual que el promedio del curso?

```
notas>=mean(notas)
```

```
[1]  TRUE  TRUE FALSE FALSE FALSE  TRUE  TRUE
```

```
table(notas>=mean(notas))
```

```
FALSE  TRUE
      3     4
```

```
sum(notas>=mean(notas))
```

```
[1] 4
```



Solución no vectorizada (no recomendada)

```
media=mean(notas)
cont=0
for(i in notas){
  if(i>=media){cont=cont+1}
}
cont
```

```
[1] 4
```



Ejercicio

! Importante

Un grupo de estudiantes obtuvo las notas que se encuentran en el vector `nota` y su sexo se encuentra en el vector `sexo`

- Cuántas mujeres y cuantos hombres hay?
- Cuál es la nota promedio de mujeres y hombres?
- Escribir código R que me diga cuántos estudiantes tienen una nota mayor o igual que el promedio de nota del curso.
- Escribir código R que me diga cuantos estudiantes mujeres (`sexo=="F"`) tienen una nota mayor o igual que el promedio de nota de las mujeres.



Ejercicio (Cont)

```
set.seed(1234)
sexo=sample(c("M","F"),500,replace=TRUE)
n1=sum(sexo=="M")
n2=500-n1
nota=numeric(500)
nota[sexo=="M"]=round( rnorm(n1,3.5,0.5), 1)
nota[sexo=="F"]=round( rnorm(n2,3.5,0.5), 1)
```



Solución

```
sum(sexo=="M")
```

```
[1] 247
```

```
length(sexo)-sum(sexo=="M")
```

```
[1] 253
```

```
sum(sexo=="F")
```

```
[1] 253
```

```
c("Femeninos"=sum(sexo=="F") , "Masculinos"=sum(sexo=="M"))
```

Femeninos	Masculinos
253	247



Comparación de números no enteros (reales o floats)

Comparar no enteros usando `==` es problemático. Todos los números que hemos tratado hasta ahora son números de punto flotante (`float`). Eso significa que se almacenan en la forma $a \times 2^b$, para dos números a y b . Dado que todo este formato debe almacenarse en 32 bits, el número resultante es solo una aproximación de lo que realmente se desea. Esto significa que los errores de redondeo a menudo se introducen en los cálculos y las respuestas esperadas pueden ser incorrectas.



Comparación de números no enteros: ejemplos

Observe el siguiente ejemplo donde se compara $(\sqrt{2})^2$ con 2, sabemos por matemáticas básicas que esa igualdad es cierta, sin embargo ...

```
sqrt(2) ^ 2 == 2
```

```
[1] FALSE
```

```
sqrt(2) ^ 2-2 #
```

```
[1] 4.440892e-16
```

ese valor que es casi cero es el error de redondeo.



Comparación de números no enteros: ejemplos

R también proporciona la función `all.equal()` para verificar la igualdad de números. Esta función proporciona un nivel de tolerancia (por defecto, alrededor de 1.5×10^{-8}), de modo que se ignoran los errores de redondeo menores que la tolerancia:

```
all.equal(sqrt(2) ^ 2, 2)
```

```
[1] TRUE
```

```
all.equal(sqrt(2) ^ 2, 3)
```

```
[1] "Mean relative difference: 0.5"
```



Uso de all.equal

Si los valores a comparar no son los mismos, `all.equal` devuelve un informe sobre las diferencias. Si necesita un valor VERDADERO o FALSO, debe incluir la llamada a `all.equal` en una llamada a `isTRUE`

```
isTRUE(all.equal(sqrt(2) ^ 2, 3))
```

```
[1] FALSE
```

```
isTRUE(all.equal(sqrt(2) ^ 2, 2))
```

```
[1] TRUE
```



Comparación de cadenas

También podemos usar `==` para comparar cadenas. En este caso, la comparación distingue entre mayúsculas y minúsculas, por lo que las cadenas deben coincidir exactamente. También es teóricamente posible comparar cadenas usando mayor o menor que (`>` y `<`):

```
c("Can", "you", "can", "a", "can", "as", "a", "canner", "can", "can", "a",  
) == "can"
```

```
[1] FALSE FALSE  TRUE FALSE  TRUE FALSE FALSE FALSE  TRUE  TRUE FALSE FALSE
```



Comparación de cadenas (Cont)

```
c("A", "B", "C", "D") < "C"
```

```
[1] TRUE TRUE FALSE FALSE
```

```
c("a", "b", "c", "d") < "C"
```

```
[1] TRUE TRUE TRUE FALSE
```



Asignando y nombrando variables

Los nombres de las variables pueden contener letras, números, puntos y guiones bajos, pero no pueden comenzar con un número o un punto seguido de un número.

No se permiten palabras reservadas como `if` y `for`. En algunos lugares, se permiten letras que no sean ASCII, pero para la portabilidad del código es mejor ceñirse a la `a` a la `z` (y `A` a la `Z`). La página de ayuda `?make.names` ofrece detalles precisos sobre lo que está permitido y lo que no.



Asignando y nombrando variables

```
.x5<-4  
#?make.names  
x<-1:5  
y=6:10  
#ls()  
print(.x5)
```

```
[1] 4
```



Asignando y nombrando variables

Ahora podemos reutilizar estos valores en nuestros cálculos posteriores:

```
x + 2 * y - 3
```

```
[1] 10 13 16 19 22
```



Asignando y nombrando variables

También podemos hacer asignaciones globales usando `<< -`. Esto se usa en la programación de funciones para crear una variable disponible en cualquier lugar, lo cual estudiaremos más adelante.



Asignando y nombrando variables

Observe que cuando asigna una variable, no ve el valor que se le ha dado. Para ver qué valor contiene una variable, simplemente escriba su nombre en el símbolo del sistema

```
z<-x + 2 * y - 3
```

```
z
```

```
[1] 10 13 16 19 22
```



Asignando y nombrando variables

Si desea asignar un valor e imprimirlo todo en una línea, tiene dos posibilidades.

- En primer lugar, puede poner varias declaraciones en una línea separándolas con un punto y coma (;) .
- En segundo lugar, puede envolver la tarea entre paréntesis, (). En los siguientes ejemplos, `rnorm` genera números aleatorios a partir de una distribución **normal** y `rlnorm` genera ellos de una distribución **lognormal**



Asignando y nombrando variables

```
## primera forma  
z<-x + 2 * y - 3;z
```

```
[1] 10 13 16 19 22
```

```
## segunda forma  
( z<-x + 2 * y - 3 )
```

```
[1] 10 13 16 19 22
```



Números especiales

Para ayudar con la aritmética, R admite cuatro valores numéricos especiales: Inf, -Inf, NaN y NA

- Inf infinito positivo
- -Inf infinito negativo
- NaN es la abreviatura de Not-a-Number “no es un número” y significa que nuestro cálculo no tenía sentido matemático o no se pudo realizar correctamente
- NA es la abreviatura de Not Available “**no disponible**” y representa un valor perdido, un problema demasiado común en el análisis de datos.



Cálculos con faltantes

En general, si nuestro cálculo implica un valor faltante, entonces también faltarán los resultados

```
5/0
```

```
[1] Inf
```

```
log(0)
```

```
[1] -Inf
```

```
sqrt(-2)
```

```
[1] NaN
```

```
Warning message in sqrt(-2):  
"Se han producido NaNs"
```



Ejemplo de NA

```
#setwd("~/Library/Mobile #Documents/com~apple~CloudDocs/cursos/estadist#ica")
datos=read.csv2("./datos/ejemplo.csv")
```

datos

	x	y
1	2	3
2	4	6
3	1	1
4	5	NA
5	6	7



Aritmética con Inf e -Inf

```
c(Inf + 5, Inf - 1000, Inf - Inf, Inf+Inf)
```

```
[1] Inf Inf NaN Inf
```



```
x=Inf + 5  
y=Inf-1000  
x-y
```

```
[1] NaN
```

```
c(10000 / Inf, Inf / 1, Inf / Inf)
```

```
[1] 0 Inf NaN
```

```
c(sqrt(Inf), sin(Inf))
```

```
[1] Inf NaN
```

```
Warning message in sin(Inf):  
"Se han producido NaNs"
```



```
c(log(Inf), log(Inf, base = Inf))
```

```
[1] Inf NaN
```

Warning message:

"Se han producido NaNs"

```
c(NA + 1, NA * 5, NA + Inf)
```

```
[1] NA NA NA
```



Aritmética con NA y NaN

Cuando la aritmética involucra NA y NaN, la respuesta es uno de esos dos valores, pero cuál de esos dos depende del sistema:

```
c(NA + NA, NaN + NaN, NaN + NA, NA + NaN)
```

```
[1] NA NaN NA NA
```

Hay funciones disponibles para verificar estos valores especiales.



Observe que NaN y NA no son finitos ni infinitos, y falta NaN, pero NA es un número:

```
x <- c(0, Inf, -Inf, NaN, NA)  
is.finite(x)
```

```
[1] TRUE FALSE FALSE FALSE FALSE
```



```
is.infinite(x)
```

```
[1] FALSE TRUE TRUE FALSE FALSE
```

```
is.nan(x)
```

```
[1] FALSE FALSE FALSE TRUE FALSE
```

```
s=c("M","F","F","F","M")
```

```
sum(s=="F")
```

```
[1] 3
```



```
is.na(x)
```

```
[1] FALSE FALSE FALSE  TRUE  TRUE
```

```
z<-c(3,1,NA,7,9,3,NA)
```

```
sum(is.na(z))
```

```
[1] 2
```



```
mean(z)
mean(na.omit(z))
mean(z,na.rm=TRUE)
min(z)
min(z,na.rm=TRUE)
max(z)
```



```
mean(na.omit(z))
```

```
[1] 4.6
```

```
mean(z,na.rm=TRUE)
```

```
[1] 4.6
```



```
min(z)
```

```
[1] NA
```

```
min(z,na.rm=TRUE)
```

```
[1] 1
```

```
sum(is.na(z))
```

```
[1] 2
```



Valores lógicos

Además de los números, el cálculo científico a menudo involucra valores lógicos, particularmente como resultado del uso de operadores relacionales ($<$, $>$, $=$, etc.). Muchos lenguajes de programación usan lógica booleana, donde los valores pueden ser VERDADEROS o FALSOS. En R, la situación es un poco más complicada, ya que también podemos tener valores perdidos, NA.



Valores lógicos (cont)

TRUE y FALSE son palabras reservadas en R: no puede crear una variable con ninguno de esos nombres (aunque las versiones en minúsculas o en mayúsculas y minúsculas como True están bien). Cuando inicia R, las variables T y F ya están definidas para usted, tomando los valores TRUE y FALSE, respectivamente. Esto puede ahorrarle un poco de escritura, pero también puede causar grandes problemas. T y F no son palabras reservadas, por lo que los usuarios pueden redefinirlas.



Valores lógicos (cont)

Esto significa que está bien usar los nombres abreviados si está trabajando en la línea de comando, pero no si su código va a interactuar con el de otra persona

- ! se usa para la negación no (not)
- & se usa para la conjunción y (and).
- | se usa para la disyunción o (or).



Valores lógicos (cont)

```
b<-3  
if(!b<0){print("b no es Negativo")}
```

```
[1] "b no es Negativo"
```

```
!is.na(z)
```

```
[1] TRUE TRUE FALSE TRUE TRUE TRUE FALSE
```

```
(x <- 1:10 >= 5)
```

```
[1] FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE
```



Valores lógicos (negación)

```
x
```

```
[1] FALSE FALSE FALSE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
```

```
# negación de x
```

```
!x
```

```
[1]  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE
```



Valores lógicos (aplicación)

```
## TRUE donde se encuentre un par  
(y <- 1:10 %% 2 == 0)
```

```
[1] FALSE TRUE FALSE TRUE FALSE TRUE FALSE TRUE FALSE TRUE
```

```
# Seleccionar los pares en b  
b<-1:10  
b[y]
```

```
[1] 2 4 6 8 10
```

```
b[!y]
```

```
[1] 1 3 5 7 9
```



Valores lógicos (aplicación)

```
z<-c(6,1,4,7,9,8,1,9,7)
# los números pares en z
z[z%%2==0]
```

```
[1] 6 4 8
```

Ejercicios

Considere el vector z anterior y escriba código que nos permita saber

- ¿En qué posición están los pares?
- ¿Cuál es el mínimo valor de z?
- ¿Cuántas veces está el mínimo en z?
- ¿En qué posición(es) está(n) el (los) mínimo(s)?
- Repita los tres anteriores pero con el máximo



Función which

La función `which` recibe un vector lógico y regresa las posiciones donde se encuentran los valores lógicos `TRUE`

```
which(c(TRUE,FALSE,TRUE,TRUE,FALSE))
```

```
[1] 1 3 4
```



Aplicaciones de which

Saber donde se encuentran los numeros pares en un vector

```
z<-c(6,1,4,7,9,8,1,9,7)  
which(z%%2==0)
```

```
[1] 1 3 6
```



Aplicaciones de which

Saber en qué posición está un número particular, por ejemplo saber la posición de 9 en z

```
which(z==9)
```

```
[1] 5 8
```



Funciones `which.max` y `which.min`

```
which.max(z)
```

```
[1] 5
```

```
which.min(z)
```

```
[1] 2
```

```
which(z==max(z))
```

```
[1] 5 8
```

```
which(z==min(z))
```

```
[1] 2 7
```



Operadores lógicos

Estudiaremos los operadores lógicos and (&), or (|) y la negación lógica not (!)

Operador &

x

```
[1] FALSE FALSE FALSE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
```

y

```
[1] FALSE  TRUE FALSE  TRUE FALSE  TRUE FALSE  TRUE FALSE  TRUE
```

operador &

x & y

```
[1] FALSE FALSE FALSE FALSE FALSE  TRUE FALSE  TRUE FALSE  TRUE
```



Operador |

```
# operador |  
x
```

```
[1] FALSE FALSE FALSE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
```

```
y
```

```
[1] FALSE  TRUE FALSE  TRUE FALSE  TRUE FALSE  TRUE FALSE  TRUE
```

```
x | y
```

```
[1] FALSE  TRUE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
```



Operaciones con los tres valores lógicos

```
library(knitr)
kable(OpLog, format = "html", table.attr = "class='striped'")
```

x	y	not.x	not.y	x_or_y	x_and_y
TRUE	TRUE	FALSE	FALSE	TRUE	TRUE
FALSE	TRUE	TRUE	FALSE	TRUE	FALSE
NA	TRUE	NA	FALSE	TRUE	NA
TRUE	FALSE	FALSE	TRUE	TRUE	FALSE
FALSE	FALSE	TRUE	TRUE	FALSE	FALSE
NA	FALSE	NA	TRUE	NA	FALSE
TRUE	NA	FALSE	NA	TRUE	NA
FALSE	NA	TRUE	NA	NA	FALSE
NA	NA	NA	NA	NA	NA



Cálculo de estadísticas en presencia de datos faltantes

- Por defecto, cuando un vector tiene datos faltantes, R regresa NA
- Ejemplo con la media

```
edad=c(20,18,15,19,NA,23,NA)  
mean(edad)
```

```
[1] NA
```

```
mean(na.omit(edad))
```

```
[1] 19
```



Máximo y mínimo

```
max(edad, na.rm=TRUE)
```

```
[1] 23
```

```
min(edad)
```

```
[1] NA
```



¿Cuántos faltantes hay?

```
sum(is.na(edad))
```

```
[1] 2
```



Imputación

- En algunas aplicaciones nos piden que reemplacemos los datos faltantes por la media o mediana de los datos disponibles, esto se conoce como Imputación

```
edad[5]=median(edad,na.rm=TRUE)
edad[7]=median(edad,na.rm=TRUE)
edad
```

```
[1] 20 18 15 19 19 23 19
```

```
edad=c(20,18,15,19,NA,23,NA)
edad
```

```
[1] 20 18 15 19 NA 23 NA
```

```
edad[is.na(edad)]=median(edad,na.rm=TRUE)
edad
```

```
[1] 20 18 15 19 19 23 19
```



Funciones any() y all()

Otras dos funciones útiles para tratar con vectores lógicos son any() y all(), que devuelven TRUE si el vector de entrada contiene al menos un valor TRUE o solo valores TRUE, respectivamente:

```
no_trues<-c(FALSE,FALSE,FALSE)
algun_true<-c(FALSE,TRUE,FALSE)
todos_true<-c(TRUE,TRUE,TRUE)
```

```
any(no_trues)
```

```
[1] FALSE
```

```
any(algun_true)
```

```
[1] TRUE
```

```
any(todos_true)
```

```
[1] TRUE
```



```
all(no_trues)
```

```
[1] FALSE
```

```
all(algun_true)
```

```
[1] FALSE
```

```
all(todos_true)
```

```
[1] TRUE
```



Aplicación: averiguar si hay algún par

```
# averiguar si hay algún par  
v=c(3,5,7,9,6)  
v%%2==0
```

```
[1] FALSE FALSE FALSE FALSE  TRUE
```

```
any(v%%2==0)
```

```
[1] TRUE
```

```
if(any(v%%2==0)){  
  cat("Hay algún número par en el vector  ")  
}else{  
  cat("No hay número par en el vector  ")  
}
```

Hay algún número par en el vector



Aplicación: averiguar si todos son impares

```
# averiguar si todos son impares  
v=c(3,5,7,9,8)  
v%%2!=0
```

```
[1] TRUE TRUE TRUE TRUE FALSE
```

```
all(v%%2!=0)
```

```
[1] FALSE
```

```
any(no_trues)
```

```
[1] FALSE
```

```
any(algun_true)
```

```
[1] TRUE
```

```
any(todos_true)
```

```
[1] TRUE
```



```
#log(-3)
#sqrt(-4)
x=c(4,7,-4,1)
log(x)
```

```
[1] 1.386294 1.945910      NaN 0.000000
```



```
edad=c(20,18,15,19,NA,23,NA)  
any(is.na(edad))
```

```
[1] TRUE
```

```
n=1:5  
any(is.na(n))
```

```
[1] FALSE
```

```
all(!is.na(n))
```

```
[1] TRUE
```



```
z<-c(5,7,1,7,3)
# hay algun par en z ?
any(z%%2==0)
```

```
[1] FALSE
```



```
all(no_trues)
```

```
[1] FALSE
```

```
all(algun_true)
```

```
[1] FALSE
```

```
all(todos_true)
```

```
[1] TRUE
```

```
# todos los elementos de z son impares ?
```

```
all(!z%%2==0)
```

```
[1] TRUE
```



Cuántos 6 tiene el vector a

```
a=c(6,9,7,3,6,7,9,6,3,6,6,7,1,9,1)
```

```
a==6
```

```
[1] TRUE FALSE FALSE FALSE TRUE FALSE FALSE TRUE FALSE TRUE TRUE FALSE
[13] FALSE FALSE FALSE
```

```
sum(a==6)
```

```
[1] 5
```

```
a>=6
```

```
[1] TRUE TRUE TRUE FALSE TRUE TRUE TRUE TRUE FALSE TRUE TRUE TRUE
[13] FALSE TRUE FALSE
```

```
sum(a>=6)
```

```
[1] 11
```

```
a<6+2
```

